

You Don't Run a Project. You Build an Asset.

Lens 2 of 3 — how the work runs, gets governed, gets defended, and becomes an asset

By Robert Paddock · idigdata

After thirty years inside business-system transformations, I have learned to distrust any program that can only explain itself at the steering-committee level.

The deck may be clean. The vendor plan may look orderly. The executive mandate may be real. None of that tells me whether the transformation can survive contact with the work.

The work lives lower.

It lives in item numbers people have memorized for twenty years. It lives in the three-way match exception nobody can make go away. It lives in a spreadsheet an analyst built before the current controller arrived. It lives in a standard-cost rollup that looks financial but is really master data, manufacturing routing, labor assumption, machine time, and human judgment braided together. It lives in approval limits, cutover sequencing,

change resistance, shop-floor data, audit trails, and the CFO's question: what are we actually building here, and can we defend it as an asset?

That is the mechanics layer.

[Article 1](#) made the people point: you do not buy a transformation; your people build one. This article makes the mechanics point. The people cannot build what the program cannot run. A transformation is not installed by a vendor. It is run from the operator's bridge until the business can run it itself.

The hopeful part is that the mechanics are knowable. They are not glamorous, but they are visible to anyone willing to look at the work instead of the deck. Again and again, the thing that first presents as a workflow problem, a vendor problem, a training problem, or a reporting problem turns out to be a data problem wearing an operational costume.

Run It From The Bridge

A transformation has to be run as one integrated operating practice.

Project management makes the work visible. Change management makes the work land. Agile-fall discipline decides which work can iterate and which work must be sequenced. Command-and-control gives the client-side program the authority to halt, redirect, or refuse vendor motion when the business reality is not ready.

Those cannot be four parallel workstreams.

Parallel workstreams look mature on a program chart and decay in the room. The PMO is tracking tasks the operators cannot absorb. The change-management lead is hearing resistance the sprint plan does not reflect. The vendor is pushing configuration forward because its lane is green. Finance is still waiting for the definition that determines

whether the downstream report is usable. Everyone is working. The transformation is drifting.

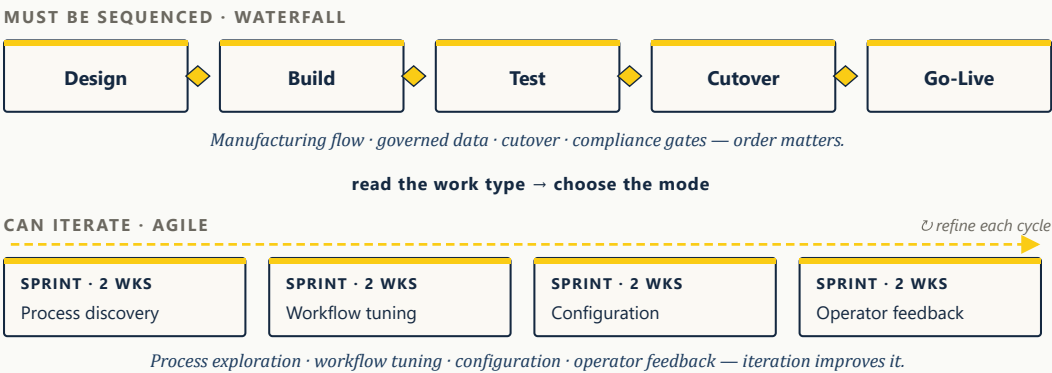
The integrated version is different.

The cadence is real. I like a thirteen- or fourteen-day sprint because it forces a program to show its work quickly. Sprint one starts when the engagement starts, not after a ceremonial mobilization period. Early sprints do not pretend the hard stuff is already known. They surface it.

Pure agile cannot carry a manufacturing flow, a governed data model, a cutover sequence, or a compliance gate where order matters. Pure waterfall cannot carry process discovery, workflow tuning, operator feedback, or the surprises that always appear when people finally see their work made visible. Most programs pick a religion. Real transformations switch modes by work type.

Agile-fall — the work decides the mode

Iterate what can iterate. Sequence what must be sequenced.



SWITCH MODES BY WORK TYPE

Most programs pick a religion. Real transformations switch modes by work type — and the work they got wrong drifts.

Agile is the cadence. Waterfall is the trajectory.

Agile is the cadence; waterfall is the trajectory. The work decides the mode.

Approval mechanics matter here more than most teams admit. It is not enough to ask who approves a purchase order, a customer setup, a vendor change, or a pricing override. The program has to ask the next question: at what limit, under what conditions, with what evidence, and who owns the exception? Approval limits are not administrative trivia. They are governance made executable.

The same is true of integrated business planning. Operators do not get to reconcile demand, supply, finance, and operations once a month in a static meeting and call it control. The business needs current inputs from every function against one governed version of reality. When Supply Chain changes demand, Finance needs to know what happened to working capital. When Operations changes a production assumption, Sales needs to know what can still be promised. When the forecast shifts, the business needs to see the operating consequence before the month closes.

That is why the program has to sit above the vendor lanes. Vendors are useful. They know their tools. They can configure, integrate, migrate, test, and advise inside their domains. But the transformation cannot be owned by the vendor stack, because the hard calls cross the stack. The operator has to own the bridge.

The Small Mechanics That Break Big Programs

The fastest way to understand a transformation is to inspect the small mechanics everyone wants to skip.

Item numbers are a perfect example.

In legacy companies, item numbers often started as a "smart scheme." Someone tried to embed meaning into the number itself. The number carried vendor, color, size, use, category, or some other schema. An operator could look at the item number and know what it was. At the time, that felt practical.

Then the company grew.

New attributes appeared that the original scheme never reserved room for. A two-character vendor code such as "SA" worked until a second vendor arrived that also wanted to be "SA." Now the business has "SAx," a patch on a patch. New employees interpreted the scheme differently. The old convention fractured. The number still looked smart, but the intelligence had already leaked out of it.

Then comes the proposed fix: renumber the items.

That is where the transformation learns what the item master really is.

Third-party services may be locked into the existing number. Warehouse people may have memorized it. Changing the number moves their cheese in a very real way. Operators may have old custom software with item numbers hard-coded in places nobody has inventoried. Multiple departments now have to agree what the new item standard should be, because the item does not belong to one department. It touches purchasing, receiving, manufacturing, inventory, costing, sales, finance, reporting, and customer service.

This is why the right answer is so simple and so hard: stop making the ID smart. It should no longer be a smart scheme, because it turns out that's dumb. Make the item number sequential and let the system's attribute and description fields carry the load.

That sentence is master data management in its purest form. Meaning belongs in governed attributes, not in the key.

The same pattern shows up in three-way match. The invoice, the purchase order, and the receipt are supposed to agree. When they do not, people talk about AP efficiency, buyer discipline, vendor behavior, or receiving process. Those may all be involved. But in mid-market operators, a stubborn exception pattern often traces back to vendor-master

mismatch, item-master mismatch, unit-of-measure mismatch, entity-boundary mismatch, or a receiving event that is not represented the same way across systems.

The exception is not just an AP problem. It is data governance showing up as controller time.

Standard product costing has the same trap. A manufacturer with three acquired entities may have three item masters, three cost-rollup methods, three routing assumptions, and three versions of what "standard" means. The consolidated variance looks like a financial result. Underneath, it may be a master-data artifact. Finance cannot interpret the variance until the business can trust the item, BOM, route, labor, machine time, overhead, and rollup logic that produced it.

Shadow IT is another version of the same truth.

Nearly every operating company has an unofficial decision layer. Spreadsheets. Analyst-owned SQL. A PowerApp someone built because the ERP could not do the work. A report exported every Monday, adjusted by hand, and trusted more than the official dashboard. A workbook whose most important tab has a name like "do not touch."

The answer is not to shame it or rebuild it blindly. The answer is find, classify, and absorb.

Find it: where is the work really being decided? Classify it: is this workaround protecting the business, creating risk, or both? Absorb it: migrate the logic into governed workflow and governed data where it belongs. Do not rebuild the spreadsheet because it is ugly. Understand what it knows. Then decide what the business should own.

These mechanics are unglamorous. They are also where transformation becomes real. If the program cannot govern item meaning, match exceptions, standard cost, and the shadow decision layer, the steering deck is not describing the business. It is describing a wish.

Manufacturing Makes The Boundary Visible

Some mechanics are universal across operating companies. Some are not.

The IT/OT boundary belongs in the manufacturing and plan-to-make conversation. It should not be universalized.

In a manufacturer, shop-floor reality has to cross into the business systems. Machines, PLCs, sensors, SCADA, MES, quality systems, WMS, production schedules, labor reporting, and ERP do not all live under one owner or one clock. Operations cares about the line running now. IT cares about integration, security, reliability, and recoverability. Finance cares about cost accounting, inventory valuation, variance, and the general ledger. Compliance cares about traceability.

That boundary is not an abstraction. It determines whether production data can become financial truth.

A multi-level bill of materials is not just an engineering record. It is the structure that tells the business what a product is made of, what substitutions are allowed, what scrap matters, what quality checks attach, and what costs roll up. A routing is not just an operations document. It is the path by which labor, machine time, and overhead become cost. A work-center master is not just a production setting. It is a financial assumption waiting to be tested by the floor.

When those records are wrong, the error travels.

The floor may still make the product. The shipment may still leave. The customer may still be served. But the margin is suspect. The inventory value is suspect. The variance is suspect. The GL receives numbers that look official because they passed through an enterprise system, but official is not the same as true.

That is why manufacturing transformations cannot treat the shop floor as a downstream integration detail. The data crossing from operational technology into business systems has to be governed as part of the transformation from the beginning. Different owners, different security demands, different timing requirements, and different consequences all meet there.

If the transformation gets that boundary right, the business can see production reality in financial language. If it gets it wrong, the business gets beautifully integrated uncertainty.

The Structural Fork

Every transformation eventually faces a fork.

One path is ERP-to-ERP migration. Move from one vendor center of gravity to another. Let the implementation partner shape the program around the destination platform. Accept the vendor's data model as the operating model. Treat business difference as fit-gap. Treat governance as a migration task. Declare victory at go-live.

Sometimes that path is the right technical move. But it is not, by itself, a business-owned transformation.

The other path is data sovereignty — the company, not the ERP vendor, owns the definitions and the decision rights. The ERP stays important, but it is not the owner of the transformation. The operator owns the common meaning of customer, vendor, item, product, location, project, cost, revenue, approval, and decision. Applications sit in their domains. Vendors contribute inside their lanes. The data model, the governance cadence, the decision record, and the workflow logic become the operator's to keep.

Master data management is the floor, not a workstream. A data council that cannot decide is a meeting. A data council with decision rights, executive cover, and business

ownership is a transformation instrument. Sales does not get a private definition of customer because it helps the pipeline. Finance does not get a private definition of revenue because the old reports were built that way. Operations does not get to keep item meaning embedded in tribal code because changing it is painful. The business decides what the enterprise meaning is, records the decision, and lives with the consequence.

Fit-to-standard belongs here too. The right question is not "Can the new system mimic every old behavior?" It is "Which differences are genuine operator advantage, and which exist only because the old system forced the business to work around it?" A company that keeps every accidental difference carries the old system into the new one. A company that flattens every real difference loses the operating knowledge that made it valuable. The discipline is knowing which is which.

The vendor-partner trap is structural: partners are paid and organized to deliver their lane, not to own the business result after everyone leaves. They can be excellent and still be the wrong organism to run the transformation.

The trap has a quieter cousin — the customizations a company has hobbled together over the years. Some are junk. Some are the secret sauce: genuinely good logic and workflows the business actually runs on. The discipline is to find that secret sauce early and decide about it on purpose — own the endpoints and define the integration path, or price in phasing it out and rebuilding it inside the new stack. What you cannot do is carry an opaque dependency into the new world and hope it holds.

I learned that the hard way. I once ran a transformation I was proud of, built in part on excellent IP from a third-party strategic partner. A year after go-live, that partner was acquired, and the acquirer forced its customers onto the merged stack. We spent the next stretch in a hasty refactor around endpoints and custom code we should have mapped and owned in the first weeks — instead of scrambling for a new solution after the business was already live. The lesson stuck: a dependency you do not own is a

dependency that can move on you. Own the endpoints, or plan their exit, before go-live, not after.

The public record keeps making the broader point. When a large operator's platform migration goes badly, it does not stay an IT story. It becomes an earnings story, a customer story, and a board-governance story — the kind that surfaces in public disclosures and shareholder litigation, not just in postmortems. Lamb Weston's troubled S/4HANA implementation, still in active litigation, is the reminder operators reach for: what the market punished was not framed as a technical failure, it was a business one. The lesson is not "pick a better vendor." It is that the operator, not the partner, has to own the outcome the business will be judged on.

There is an obvious objection here, and it points at me as much as at any vendor: an independent operator also leaves. That is exactly the point. The job is not to become the thing the business depends on. It is to leave the business owning its own data, its own definitions, its own endpoints, its own decisions — owning its way past me. A transformation run well makes its own author unnecessary. If the engagement created a new dependency, it failed, however clean the go-live looked.

The modern version of this fork matters even more because agentic systems are arriving alongside ERP, not after it. The practical question is no longer only "Which ERP will run the business?" It is also "What governed substrate will every system, workflow, report, and agent trust?" A business can keep its ERP and build the next operating layer alongside it — but only if the data floor is sovereign.

The CFO Mechanics Are Not Optional

Start with the boundary I do not cross: the CFO owns the accounting judgment. My job is to make the operating evidence clean enough that the judgment is defensible — every attributable hour, every system's status, every in-service determination traceable

to the work that produced it. With that line drawn, here is the discipline that makes a transformation fundable as an asset instead of absorbed as a cost.

Define the transformation as a capital asset from the beginning — not a stack of separate system projects, not a program reclassified as an asset at the end when someone finally asks the question. One company-wide asset build, framed that way from day one.

That does not mean the entire program is capitalized. That is the lazy claim auditors reject and CFOs distrust. It means the program has the discipline to separate what creates the asset from what supports, trains, explores, cleans up, or operates around it. The boundary gets drawn at the work, not at the workstream. Development, configuration, certain testing, directly attributable internal labor, and certain vendor integration work may belong in the capital story. Training does not become capital because it happened during the same program. Change management does not become capital because the deck says transformation. Research and feasibility are not build. Post-go-live maintenance is not placing the asset into service.

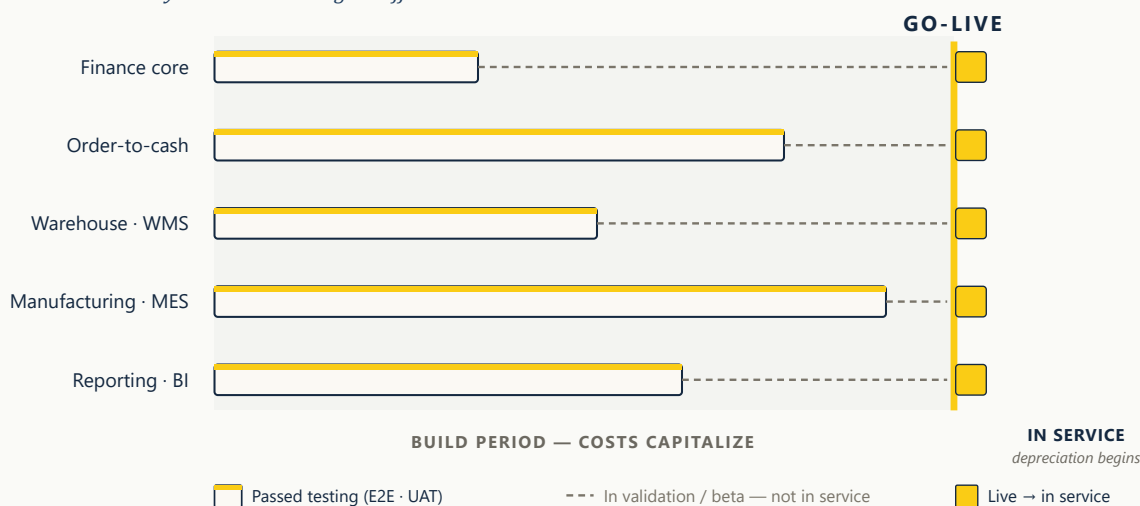
Internal labor is the lever most programs miss. The business puts real people into the build — the beehive of taskforces: finance leads, supply-chain SMEs, IT leads, process owners, data owners, controllers, planners, production experts. On a serious transformation that internal time is significant, and where it is directly attributable to building the asset and documented while the work happens, it can be capitalized. The catch is in that "while the work happens." A project financial tracker turns spend into live data instead of a quarter-end archaeology project: every entry — consultant hour, internal-labor hour, vendor work, testing, configuration, training, change management — carries a rationale, an owner, a timestamp, and a tie back to the asset or the non-capital activity it supports. Reconstruct that story after the fact and the CFO is defending a number nobody captured.

The timing is real, and it is worth showing the CFO early. Qualifying build cost that moves onto the balance sheet during the build comes off the P&L for those years; once the asset is in service, it depreciates over its useful life. The relief in the build years and the depreciation that follows are two halves of one decision — a CFO who sees the whole shape early can plan for it; a CFO who sees only the first half meets the second half as a surprise.

The part that takes real fluency is the in-service date — the moment the asset is placed into service and the depreciation clock starts. The test is not "is a system switched on." It is "has the system realized its intended use." In a transformation, a component's intended use is to run the business as part of the integrated whole. A system that has passed end-to-end and UAT testing at month twelve is tested, not operational — it has not cleared go-live validation, and the business is not yet relying on it. Even a simple system quietly consuming data and producing output someone reviews is in beta until it is hooked into the go-live core and trusted as the system of record. Components clear those validation cycles at different times across the deployment, but none is in service on its own test date, because none is running the business until the whole goes operational together. Done done is when the asset runs the business — at the coordinated go-live. That is the in-service date, and the validation record is the evidence: you can show exactly which cycle every system sits in, integrated or still in parallel, relied upon or still in test. Where a component genuinely does go into independent production early, it clocks in then — that is the CFO's call, made on the evidence, with the auditor's concurrence. But a cohesive transformation is designed so the asset goes into service as one operational whole.

One go-live. One in-service date.

Systems clear testing at different times — none is in service until the whole runs the business.



THE IN-SERVICE TEST

Tested ≠ in service. The asset is placed in service when the whole runs the business — one coordinated go-live.

Done done is when the asset runs the business.

Tested is not in service. The asset is placed in service when the whole runs the business — one coordinated go-live.

There is also a missing line item in many transformation budgets: leadership behavior under pressure. The platform cost is visible. The integrator cost is visible. The licensing and migration costs are visible. What is rarely costed is the meeting where the leader holds the line when every function wants its old definition back, the first failure where the room needs calm instead of blame, the decision to stop a vendor lane because the business is not ready to absorb what is being pushed. Those are not accounting entries. They are asset-protection moments, and they are as load-bearing as any line that is.

The board does not fund a transformation because someone has a better implementation vocabulary. It funds it because the business is building something it can carry: governed data, controlled workflows, defensible decisions, traceable spend, cleaner close mechanics, fewer avoidable exceptions, and a platform for future change. The CFO can defend that only when the mechanics produce the evidence.

Compliance Is A Companion From Day One

Compliance should not be the banner over the transformation. It should be the plumb line through it.

The first serious conversations should include the CFO and the Compliance Officer, because they carry the consequences. If the number is wrong, Finance carries it. If the control fails, Compliance carries it. If a decision cannot be explained, Legal, Audit, Finance, and Operations all feel it in different ways.

That does not mean compliance slows the work into ceremony. It means auditability is designed into the work while the work is being built. Every master-data decision has an owner. Every approval limit has a rule and an exception path. Every workflow change has a rationale. Every test produces evidence. Every system change is traceable. Every consequential decision is defensible by the people who made it.

None of this is really about installing software. It is about digging down — to the data, the definitions, the decisions, and the workflows — until the business can see itself clearly enough to govern itself: making disagreement visible, resolving it at the right altitude, and preserving the decision so the business does not have to rediscover it every quarter. The practice is named for exactly what it produces.

The Turn Toward Production Agentics

Production agentics does not lower the bar. It raises it.

An agent moves faster than a report, a workflow queue, or an analyst checking a spreadsheet. That speed is only worth anything when the agent acts on governed data, inside bounded authority, with a traceable decision path and a human owner for the calls that matter. If the customer master is disputed, the agent automates the dispute. If

the item number still hides tribal meaning nobody trusts, the agent scales the confusion. If approval limits are informal, the agent guesses where authority lives. If the shadow spreadsheet is the real decision layer, the agent learns from something the business cannot govern.

You cannot automate dysfunction. You can only make it move faster.

That is why the mechanics come before the agentics. The same work that makes a transformation hold — governed master data, owned workflows, decision records, audit trails, finance discipline, and a client-owned operating fabric — is the work that makes production agents safe enough to matter.

Build the asset first. Then the next layer has something solid to stand on.



Bring the real situation. If this is the build you are trying to run, start a conversation at idigdata.com.